Comparison and Improvement of Two Methods for Reconstruction of a Graph from Signals

Jinglei Shi, *Student Member, IEEE,* Yihong Xu, *Student Member, IEEE,* and Bastien Pasdeloup, *PH.D. Student, Tutor, TELECOM BRETAGNE*

***Abstract*—The construction of the graph structure plays a crucial role in the graph based data treatment and analysis, particularly in the emerging field of graph signal processing. However, the structure of the graph is not always known and the structure of the graph is not straightforward given signals received or collected by system. In this article, we will introduce two methods for learning graphs structures from signal observations respectively under smoothness prior and according to the conditional independence. Then, we will combined the principle idea of these two articles, more precisely, we will reuse the principle idea of the first method into the structure of the second one. The result of this combination is that we have got a much more efficient method with less time of calculation and it has a relatively good performance when the size of graph is strictly bigger than 6.**

***Index Terms*—Laplacian matrix, Matlab, graph learning, signal processing on graphs, structure construction.**

I. Introduction

A graph is a generic kind of data structure which is useful for describing relations among data in numerous fields or applications. We find examples of graph signals in many different engineering and science fields. In the research of different sections of brain functions, a graph can describe distinct functional regions of cerebral cortex[1], in networks, a graph is often used for imitating the spread of a disease[2], the movement of human and goods transportation[3], and a graph can be used in social network for diffusing information more efficiently[4]. The issue in these fields is that we presume that the graph structure is a priori known, so what is done is detecting signal by sensors or inferring some probable signals from observations. However in some circumstances, the structure of the graph is totally unknown or in some applications, a natural graph is not easy to define. Therefore, in these scenarios, we need to learn the graph topology from the observed data. The main challenge in the issue of graph learning is to use the relationship between the observed signals and the graph structure, and the work of construction must satisfy some predefined models, or have certain criteria to judge the quality of the constructed graph.

In the paper, we focus particularly on two methods for constructing graph from the observed signals: Graph Learning for Smooth Signal Representation (GL-SigRep)[5] and the estimation of the graph structure from the conditional independence(ABiGlasso)[6]. The two methods proceed differently: the method GL-SigRep focuses on the family of signals that are smooth on graphs(In other words, the observed signals are expected to be smooth on an unknown graph topology, the constructed graph will make the observed signal have a decent smoothness on the graph); And the method AbiGlasso is initially used to estimate the structure of a network of sensor.It is often thought that in a network of sensors, some sensors may have a tight relationship and some have nearly no relationship, to judge this relationship, from the mathematical perspective, the notion of conditional dependence is introduced, which means that for any two sensors in the network, after knowing the information captured by the rest of sensors. An estimation from these information can provide judgment the dependence of the two sensors, therefore, from the point of view of graphs, in the network of sensors, each sensor can be considered as a node in a graph, the value of each node is the signal captured by sensor, if there is a high dependence between two sensors, in the graph, an edge exists between these two corresponding nodes.

This document is organized as follows:

In section II, we outline some fundamental notions of the domain graph and some of mathematical basics which will be beneficial to have a deeper understanding of these two methods in the introduction.

In section III, we will give a relatively simple introduction about the method Graph Learning for Smooth Signal Representation (GL-SigRep) and the method Estimation of the graph structure from the conditional independence(ABiGlasso), including their principles, procedures and expecting results.

In section IV, their work will be implemented in our article and some initialization work will be talked about.

In section V, the results and comparison of time complexity will be presented and in the conclusion section, we will discuss the results and summarize the advantages and disadvantages of these methods. Finally, in the perspectives part, we will give some directions for possible improvements of the method, and some perspectives about the signal process on graph.

II. Basics

For a better comprehension of this article, we would like to introduce some definitions as follows:

Definition 1.1. Graph: an ordered pair G = (V, E) comprising a set V of vertices, nodes or points together with a set E of edges, arcs or lines, which are 2-element subsets of V.

Definition 1.2. Graph signals: signals defined on the vertex set V of a weighted and undirected graph G where the vertices of the graph represent the entities and the edge weights reflect the pairwise relationships between these entities.

Mathematically, a graph structure can be represented by a symmetrical matrix named adjacency matrix A.

Definition 1.3. Adjacency matrix: an adjacency matrix is a means of representing which vertices (or nodes) of a graph are adjacent to which other vertices. the value of the element in the i line and j row in the matrix is 0 means that no relation between the i^{st} vertex and j^{st} vertex.

Definition 1.4. Smoothness: Smoothness can describe the variation of values of graph signals, a smooth graph means that graph signals share similar signal values in the graph.

we can quantify the smoothness by the equation below :

$$s = \frac{1}{2}\sum_{i\in n}\sum_{j\in n} w_{ij}(X(i) - X(j))^2, \tag{1}$$

here w_{ij} is the weight between two nodes since we use a graph without weight, the value of w_{ij} is binary, i.e., 0 stands for no edges between them and 1 reversely.

Definition 1.5. Covariance matrix: a covariance matrix is a matrix whose element in the i, j position is the covariance between the i_{th} and j_{th} elements of a random vector (that is, of a vector of random variables).

Definition 1.6. Precision matrix: the precision matrix is the matrix inverse of the covariance matrix.

Definition 1.7. Eigenvalues and eigenvectors: In linear algebra, an eigenvector is a vector that does not change its direction under the associated linear transformation. In other wordsłif **v** is a vector that is not zero, then it is an eigenvector of a square matrix A if $A\mathbf{v}$ is a scalar multiple of **v**. This condition could be written as the equation:

$$A\mathbf{v} = \lambda\mathbf{v}, \tag{2}$$

where λ is a scalar known as the eigenvalue or characteristic value associated with the eigenvector **v**.

A graph signal can be also represented by a function $f : V \to R^n$,which assigns a scalar value to each vertex.

For further mathematical treatment, we introduce the Laplace matrix L which is a matrix representation of a graph.

$$L = D - A, \tag{3}$$

where degree matrix D is a diagonal matrix whose i^{st} diagonal element d_i is equal to the sum of the weights of all the edges incident to vertex i and A is the adjacency matrix. for the treatment of signal on graph, Laplace matrix could be a powerful tool transforming signals on the graph to the graph spectral domain, more detail will be in section III.

III. Related Work

A. The Work of Dong et al. [5]: GL-SigRep

Generally speaking, we might have more than one solution for a graph structure associated to given signals. One of the meaningful structures is based on the smoothness. Signals nodes share similar signal values, which is the case in the nature world. For example, consider a graph where the vertices represent cities and the signals are temperatures, the values are similar in the same region. In the work of Dong et al., based on this fact, they tried to get the structure of a graph by enhancing the characteristics of smoothness from the input signals. Hence they transformed the problem of finding a graph structure to a problem of finding the Laplacian matrix L of this graph maximizing smoothness.
More precisely, Dong et al.[5] designed a model to represent the received signals with

$$x = \chi h + u_x + \epsilon, \tag{4}$$

where χ is the representation matrix and h is the latent variable that controls the graph signal x, u_x is the mean of x and ϵ is the isotropic noise. In their model, they chose the representation matrix as the eigenvector matrix χ of the graph Laplacian L. They also assumed that the latent variable h follows a degenerate zero-mean multivariate Gaussian distribution with the precision matrix defined by the eigenvalues Λ of the graph Laplacian L. This assumption implies that the energy of the signal tends to lie in the low frequency components, which enhances the smoothness of the graph signal.

The algorithm is mainly based on two problems of optimization:

$$\begin{aligned} \min_{L,Y} \|X - Y\|_F^2 + \alpha\, tr\left(Y^T L Y\right) + \beta \|L\|_F^2 \\ \mathbf{s.t} \qquad tr\,(L) = n, \\ L_{ij} = L_{ji} i \neq j, \\ L \cdot \mathbf{1} = \mathbf{0}, \end{aligned} \tag{5}$$

where α and β are two regularization parameters that are positive, these two parameters can give weight to both sides of the sum operation, here we choose $\alpha = 1$ and $\beta = 100 * \alpha$ as recommended in the article Dong et al., $\mathbf{1}$ and $\mathbf{0}$ denote the constant one and zero vectors and Y is the matrix of $y = \chi h$ with χ is the eigenvector matrix, $h \in \mathbb{R}^n$ represents the latent variable that controls the graph signal X.

$$\min_{Y} \|X - Y\|_F^2 + \alpha\, tr\left(Y^T L Y\right). \tag{6}$$

Notice that the equation : (6) has the closed form solution :

$$Y = (I_n + \alpha L)^{-1} X. \tag{7}$$

where I_n represents the identity matrix of dimension n.

The algorithm changed to one problem of optimization which is more simple in terms of time calculation complexity.for more detail about the formula derivation and prove, please see Dong et al.[5].

The principle idea of this algorithm is as follows:

Algorithm 1 Graph Learning for Smooth Signal Representation (GL-SigRep)

Input: Input signal$X$, number of iterations $iter$,α,β
Output: Output signal Y, graph Laplacian L
1: **Initialization:** $Y = X$
2: **for** $t = 1, 2, ..., iter$ **do**
3: **Step to update Graph Laplacian L:**
4: Solve the optimization problem of Eq.5 to update L.
5: **Step to update Y:**
6: Solve the optimization problem of Eq.6 to update Y.
7: **end for**
8: $L = L_{iter}$, $Y = Y_{iter}$.

B. The Work of A. Costard [6]: ABiGlasso

As mentioned in the introduction, it is possible to reconstruct a graph just from the signals of each node with the help of conditional dependence. In the algorithm AbiGlasso, which is based on the algorithm Graphical lasso[7], the signal is given in form of matrix, the output representing relationship between sensors is also in form of matrix, named precision matrix, by setting a threshold, the estimation of the graph structure can be done easily, generally, the threshold is set to be zero, which means that there is an edge if the correspondent element in the matrix is greater than 0 and no edge if element is less than 0. In the implement of this algorithm, the algorithm Graphical lasso is a method for estimating a sparse precision matrix from the empiric precision matrix, the principle of algorithm Graphical lasso is to find a $\hat{K}_\lambda$ such that the formula has the greatest value:

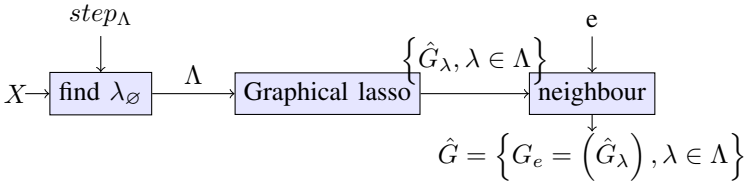
$$\hat{K}_\lambda = \arg\max_K \log(\det(K)) - tr(K\Sigma_{emp}) - \lambda \|K\|_1 , \quad (8)$$

This is the matrix which we use to estimate the structure of the graph, and λ the penalization parameter. From $\hat{K}_\lambda$, we can judge easily whether there is an edge between each of two nodes, if has k_{ij} a positive value, it means connection between node $i$and node j, and a negative value means isolation between node i and node j, λ is a parameter enforcing a degree of sparsity, and the value of λ is negatively correlated with the number of edge: when λ = 0, the graph is complete or has most of its edges, when λ arrive its upper border, there is no edge in the graph. For certain value of λ, this method may not converge, to solve this problem, an option is making a vector of λ, from minimal value to its upper border, the graph calculated from this vector ranges from complete graph to isolate graph, these graphs are initial graphs. In these initial graph, it's not definite to get at least one complete graph, so it is important to make variations on the base of these graphs, in ABiGlasso, to make such a variation, several new edges can be added in the graph or several existing edges can be also eliminated, the number of edge added or eliminated is a variable of the algorithm, when this variable is decided, with initial group of graph, after the treatment of duplicate removal and election of connected graph, a new group of graph will be generated, including some complete graphs which is referred as graph for election, to elect the best graph, a score is calculated according to the formula:

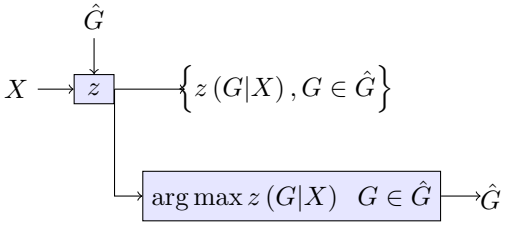
$$z\left(G|X\right) = \frac{\varphi_{\pi_{\bar{E}}, cW_{\bar{E}\bar{E}}}\left(0\right)}{V\left(G\right)}. \quad (9)$$

the explication of each item can be find in the paper of A.Costard[6] , this score can be used to choose the graph that makes the posterior probability maximum. The algorithm is established on the physic phenomenon and mathematical model, so the result may approach the most possible result, but with calculation of score, it is demanded to compare all the graph and calculate all the scores, that makes complexity enormous and an ill-suited number of edge added or eliminated can make a strange result.

Here is the steps to realize this method:



(a) Initialization Step



(b) Comparaison Step

Fig. 1: ABiGlasso algorithm

To realize algorithm ABiGlasso, there should be two steps: Initialization and comparison, and the procedure of each step is described as follows:

(1) It begins by looking for the upper boundary $\lambda_\varnothing$ of a set of penalization parameters, and the number of the edge in each graph obtained depend upon λ: for $\lambda = 0$, the graph will be complete and when λincreases, the number will decrease until zero, $\lambda_\varnothing$ is the value from which the graph obtained has no edge, the value of $\lambda_\varnothing$ is calculated by dichotomy making that the difference between $\lambda_\varnothing$ and the previous one is less that a step. The output of this step is a vector $\Lambda = \{0, step_\Lambda, 2step_\Lambda, \cdots, \lambda_\varnothing\}$.
(2) After obtaining Λ, an initial group of graph $\hat{G}$ will be generated by algorithm Graphical lasso.
(3) For each graph in $\hat{G}$, it exists a group of neighborhood in which each graph has at most e differences, those neighborhood form a new group of graph named $\hat{G}_e$, every element in $\hat{G}_e$ is distinct.
The step of comparison consists of calculating the score z of each graph in $\hat{G}_e$ from the formula:

$$\hat{G} = \arg\max z\left(G|X\right) \quad G \in \hat{G}. \quad (10)$$

And the graph with the greatest value of z is the graph wanted.

IV. CONTRIBUTION

We will implement the two algorithms and for the method ABiGlasso, a variant of algorithm will be implemented simply by changing the definition of score z, precisely, the smoothness formula will be used as score. After that, the comparison part will be presented in section V. We will compare the methods implemented in terms of time complexity and smoothness. We would like to present the relation between number of vertices with computation time and Smoothness. From this comparison, some disadvantages and advantages will be discussed.

A. GL-SigRep

Our implementation used Matlab as coding language for the reason thar it provides many mathematical tools, one of which is the CVX tool[8]. It is one of the tools served to solve optimization problems. We would like to compare our Laplacian matrix L calculated with the Laplacian matrix L from the Eq.3, therefore, we created a connected graph:

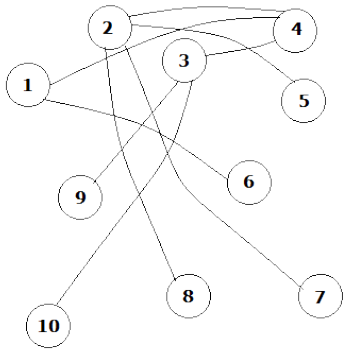


Fig. 2: Example: A connected graph

We calculated the adjacency matrix A and the degree matrix of this graph, the results are as follows:

$$A = \begin{bmatrix} 0&0&0&1&0&1&0&0&0&0\\ 0&0&0&1&1&0&1&1&0&0\\ 0&0&0&1&0&0&0&0&1&1\\ 1&1&1&0&0&0&0&0&0&0\\ 0&1&0&0&0&0&0&0&0&0\\ 1&0&0&0&0&0&0&0&0&0\\ 0&1&0&0&0&0&0&0&0&0\\ 0&1&0&0&0&0&0&0&0&0\\ 0&0&1&0&0&0&0&0&0&0\\ 0&0&1&0&0&0&0&0&0&0 \end{bmatrix} \quad D = \begin{bmatrix} 2&0&0&0&0&0&0&0&0&0\\ 0&4&0&0&0&0&0&0&0&0\\ 0&0&3&0&0&0&0&0&0&0\\ 0&0&0&3&0&0&0&0&0&0\\ 0&0&0&0&1&0&0&0&0&0\\ 0&0&0&0&0&1&0&0&0&0\\ 0&0&0&0&0&0&1&0&0&0\\ 0&0&0&0&0&0&0&1&0&0\\ 0&0&0&0&0&0&0&0&1&0\\ 0&0&0&0&0&0&0&0&0&1 \end{bmatrix}$$

Fig. 3: Graph initialization

We can get the Laplacian matix by Eq.3.

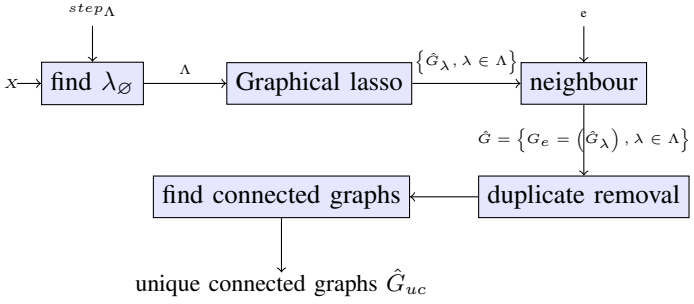
As described in Dong et al.[5], we can generate the signal X assuming that $x \in \mathbb{R}^n$ is a multivariate Gaussian signal with mean zero and covariance matrix $L^+ + \sigma_\epsilon I_n$, where L^+ is the pseudoinverse of Laplacian L ,σ_ϵ is covariance of Gaussian noise and I_n is identity matrix of n. We took 0.5 for value of σ_ϵ and generate 200 colones of values which makes signal X is a 200 colones, n rows (here $n = 10$) matrix. This signal will also be used for the following algorithms as input data. the value of L and Y will converge when value of iteration is bigger than 200, here we took $iter$ as 200.

B. ABiGlasso

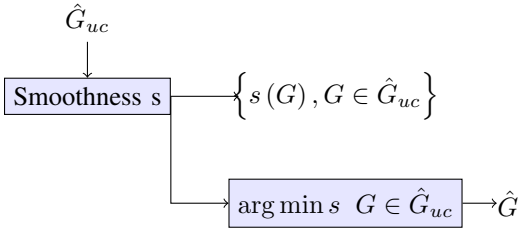
In the initialization of our method, there are four parameters as inputs, X as the signals captured with dimension 10×200, meaning 10 nodes and 200 times of sampling. And step means the step size in the vector of penalization , in our initialization, step is set to be 0.1. Then threshold is to determine whether there exist an edge between two nodes, because value to be compared is the conditional correlation, threshold is set to be 0. Finally, e is the maximal number of edge to be added or eliminated, in our implement, e takes the value 6. We use the toolbox ABiGlasso[9] from A.Costard and also the useful mathematic functions from Stanford Matlab scripts[10].

C. ABiGlasso with Smoothness

ABiGlasso with Smoothness is a method based on ABiGlasso. The score z is one of the way to quantify the quality of output graphes. But for ABiGlasso, it takes to much time(sometimes more than 2 hours depending on size of matrix) to compute, it is one of the problems of ABiGlasso which severely reduces its performance. Here, we change the definition of score simply by Smoothness, which can be described with the formula of smoothness 1, the smaller value of smoothness, the smoother the graph is. A procedure that can explain this method:



(a) Initialization Step



(b) Comparaison Step

Fig. 4: ABiGlasso with Smoothness algorithm

This schema is quite the same as figure 1 except that the score definition is changed and that some blocks are added for adjusting this change and to keep the graphs having good properties(connectivity, Smoothness, etc.). Before using Smoothness, we need to make sure that there is no non-connected graphs in the input because from the Eq. 1, we know that the result will be always the non-connected graph

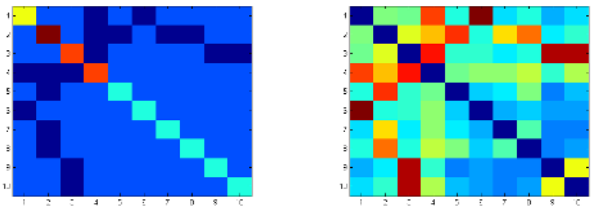
whose smoothness is smallest among the connected graphs. Our goal is to create a meaning connected graph so all the non-connected graphs will previously be removed. the operation duplicate removal is just to save the calculation time by avoiding recalculating Smoothness for the same graph many times. We used some existing mathematic functions from Stanford Matlab scripts[10].

V. Results

For each algorithm with the same signals as described before, an exact Laplacian matrix L of the figure 2 is used as an example to compare with the Lapalacian matrices L calculated by the two algorithms(GL-SigRep and ABiGlasso with Smoothness). for the rest (ABiGlasso and ABiGlasso with Smoothness) will use a 6×6 graph.

A. Result: GL-SigRep

The original output of GL-SigRep is a Laplacian matrix L, it is useful to firstly compare the output L with the exact Laplacian L of original graph. In Fig. 5, although at the first glance we cannot find the similarity of these two figures, but we can tell the shape of figure(a) by focusing on the dark color parts of figure (b). In fact, GL-SigRep has the ability to describe the relative relation between vertices of original graph. But the disadvantage is that it should pass to a threshold to transform to the binary adjacency matrix. The difficulty is that the values of calculated L change according to the times of iteration and number of signal sample X, which makes it hard to decide a constant value of threshold from all cases.

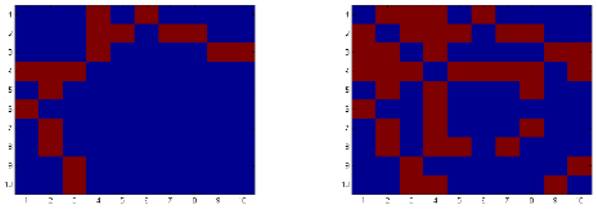


(a) Original Laplacian Matrix L in figure 2

(b) Laplacian matrix calculated from GL-SigRep

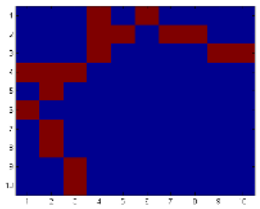
Fig. 5: Comparison Laplacian matrices L

We defined $mean$ the average value of matrices Laplacian L from GL-SigRep. If the threshold is well chosen, the adjacency matrix can be exactly the same as that of the original graph, see Fig. 6.



(a) Original adjacency matrix

(b) Adjacency matrix from GL-SigRep with $threshold = mean$

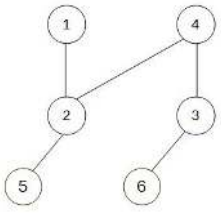


(c) Adjacency matrix from GL-SigRep with $threshold = 1.5 * mean$

Fig. 6: Comparison of different adjacency matrices

B. Result: ABiGlasso

The most serious problem is that it costs much time to calculate score z, we attempted to use the 10×10 graph, but it took us more than 2 hours to calculate and ended in breakdown of system. Therefore, we decided to use a 6×16 graph for testing its results.



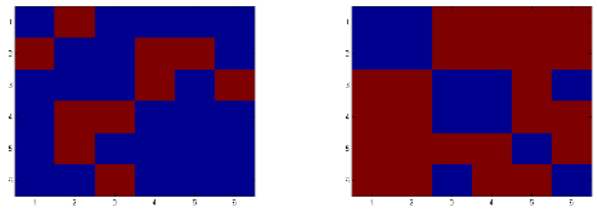
(a) Example: A connected graph N=6

$$L = \begin{bmatrix} 1 & -1 & 0 & 0 & 0 & 0 \\ -1 & 3 & 0 & -1 & -1 & 0 \\ 0 & 0 & 2 & -1 & 0 & -1 \\ 0 & -1 & -1 & 2 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix}$$

(b) Laplacian Matrix calculated from GL-SigRep

Fig. 7: Graph and Laplacian matrix

We got the result as follows:



(a) Original adjacency matrix

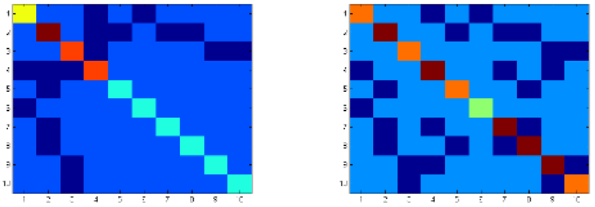
(b) Adjacency matrix from ABiGlasso

Fig. 8: Comparison of different adjacency matrices

The result is not as good as predicted, from our point of view, it's because that the signal x of 200 samples is not enough, and our assumption is that x follow a multivariate Gaussian process which works well in GL-SigRep is not suitable for ABiGlasso.

C. Result: ABiGlasso with Smoothness

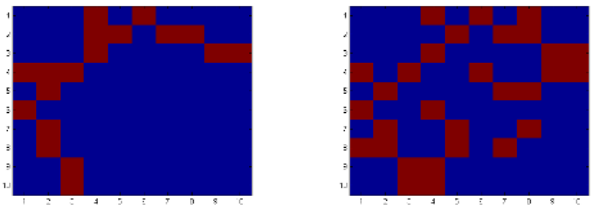
we would like firstly to compare the Laplacian matrices L:



(a) Exact L in figure:2 (b) Laplacian matrix from ABiGlasso with Smoothness

Fig. 9: Comparison of different laplacian matrices L

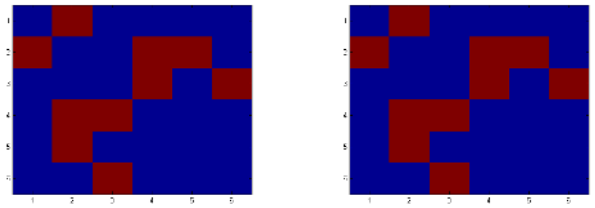
From the view of Laplacian matrices, some false values are introduced for GL-SigRep. This method has no need to solve the problem of optimization. Its computation time is much less than others.



(a) Original adjacency matrix (b) Adjacency matrix from ABiGlasso with Smoothness

Fig. 10: Comparison of different adjacency matrices

The result is as its Laplacian matrix, some false points is introduced. We retried the algorithm with lower number of vertices(N=6) and keeps other parameters unchanged(e=6, threshold=0, samples of signal=200, step=0.1), the result is satisfying and we have got the exact graph as the original one.



(a) Original adjacency matrix with $N = 6$ (b) Adjacency matrix $N = 6$ from ABiGlasso with Smoothness

Fig. 11: Comparison of different adjacency matrices

D. Error Rate

In order that we can have a more clear view of these results, we calculated their error rate by the equation below:

$$ErrorRate = \frac{1}{N^2}\sum_{i,j}^{N}\left|G_{i,j} - \hat{G_{i,j}}\right|, \qquad (11)$$

where $G_{i,j}$ is the i^{th} row j^{th} column element of the original adjacency matrix and $\hat{G_{i,j}}$ is the i^{th} row j^{th} column element of the calculated adjacency matrix.

Name	Error rate
GL-SigRep threshold = means, N=10	0.2200
GL-SigRep threshold = 1.5*means, N=10	0.0000
ABiGlasso N=6	0.7222
ABiGlasso with Smoothness N=10	0.1600
ABiGlasso with Smoothness N=6	0.0000

Fig. 12: Table: Error rate

From the table above, we can see that our method ABiGlasso with Smoothness has the best performance among the three methods in terms of adjacency matrices. Note that the result is based on signals which follows multivariate Gaussian distrubition and the pseudoinverse Laplacian matrix is added to its variance matrix. This assumption did not work well with ABiglasso that has 0.7222 of error rate. Finally, the performance of GL-SigRep depends on its threshold when we need the adjacency matrix.

E. Time complexity and Smoothness

We choose size of graph from 1 to 20 and draw the size-time curve for each algorithm, the samples of X is 200, e respectively equals to 0,1,2,3...19, $step = 0.1$, $threshold = 0$ and for GL-SigRep $iteration\,times = 200$. Through Matlab, the result is presented by different curves and every value relates to different N and we also give the average computation time:

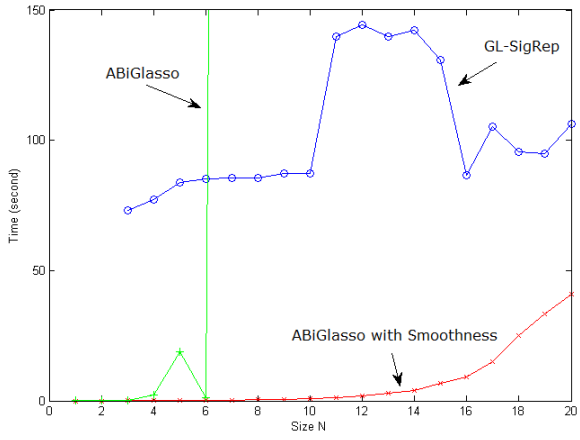


Fig. 13: Time Complexity Comparison

Name	Average computation time
GL-SigRep	92.5156 seconds
ABiglasso	361.1422 seconds
ABiGlasso with Smoothness	7.2016 seconds

Fig. 14: Table: Average computation time

When we ran this test, some points must be pointed out: we found that when the dimension of signal is smaller than or equal to 2, the cvx program can not run for GL-SigRep. Another observation is that it is too slow (more than 2 hours) to run ABiGlasso when N is larger than or equals to 7. In terms of time complexity, our work of ABiGlasso with Smoothness is the best for the reason that it simplifies the procedure of calculating score into a constant complexity calculation and it has a relatively good result in terms of error rate.

Smoothness is a key factor for GL-SigRep and ABiGlasso with Smoothness to estimate the quality of result. In this test, we used 200 samples of input signals, the value of GL-SigRep iteration times is 200, the size N of connected graphs varies from 1×1 to 20×20, we drew the Smoothness values for different size of graphs and calculated the average value of Smoothness. Note that when N is strictly smaller than 3, we could not run tool cvx for GL-SigRep and that the value of Smoothness is calculated from adjacency matrices.

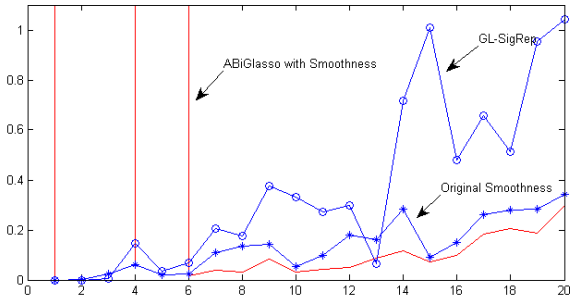


Fig. 15: Smoothness with different size N

When N is between 6 to 20, the Smoothness values are relatively close between ABiGlasso with Smoothness and original graph. The values of GL-SigRep are larger, which means that they have not a smooth graph as the original one.

We found that when N is strictly smaller than 6, the Smoothness value of ABiGlasso with Smoothness has a significant rise and then it goes back to normal value, therefore, for the average value of it we took only the 6^{th} to 20^{th} values, and for GL-SigRep, we took the 3^{th} to 20^{th} values for the reason described before.

Name	Average Smoothness
Original adjacency matrix	0.1349
GL-SigRep	0.4082
ABiGlasso with Smoothness	0.1026

Fig. 16: Table: Comparison of Smoothness

Within the reasonable area of each method, ABiGlasso with Smoothness has better values than that of GL-SigRep.

VI. Conclusion

A. Summary

In this paper ,we have presented two methods for the estimation of the graph structure from signals, the constructions are respectively from Smoothness and conditional covariance. We have showed that because of the complexity of the method ABiGlasso, the construction of the graph becomes very hard and takes a very long time to calculate. In order to simplify to time complexity, inspired by the method GL-SigRep, we choose to replace the graph quality score z with Smoothness, and then, we presented different results of estimation of graph structure in both performance and time complexity, we find that GL-SigRep can construct a graph relatively more quickly but the quality is related to the threshold, ABiGlasso with its worst time complexity, is not practical when the number of node is strictly bigger than 6, and AbiGlasso with Smoothness can estimate the graph structure much more quickly than ABiGlasso and with relatively better performance when the input signals follow multivariate Gaussian distribution whose covariance is related to Laplacian matrix l and the size of L is strictly bigger than 6.

B. Perspectives

With inspiration of method GL-SigRep and ABiGlasso, we create our own method which uses the mathematical theory of possibility to estimate the most possible structure of a graph, and then use the notion Smoothness as a criterion to select a graph, thus the graph chosen will have a structure corresponding to the real model in some application, for example: different functional segments of brain and a decent Smoothness for the signal on the graph, but this method depends on the number of initial graph, and that number have a strong relationship with number of added or eliminated edges. The next step of our idea, is to find some methods to estimate this number from the observed signals or possible structures, limiting scope of our choice and having a better result.

Acknowledgment

The authors would like to thank the direction group of ”Mineure recherche” which gives them the chance to have a brief understanding of a brand-new domain and write an IEEE standard article about it. Thanks to their tutor M. Pasdeloup, with his detailed explication and continuous support, they have finally finished their implementation of two algorithms GL-SigRep and ABiGlasso and create a variant algorithm ABiGlasso with Smoothness.

References

[1] Olaf Sporns, Giulio Tononi, and Gerald M Edelman. Theoretical neuroanatomy: relating anatomical and functional connectivity in graphs and cortical connection matrices. *Cerebral Cortex*, 10(2):127–141, 2000.

[2] JS Damoiseaux, SARB Rombouts, F Barkhof, P Scheltens, CJ Stam, Stephen M Smith, and CF Beckmann. Consistent resting-state networks across healthy subjects. *Proceedings of the national academy of sciences*, 103(37):13848–13853, 2006.

[3] David Shuman, Sunil K Narang, Pascal Frossard, Antonio Ortega, Pierre Vandergheynst, et al. The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains. *Signal Processing Magazine, IEEE*, 30(3):83–98, 2013.

[4] David Kempe, Jon Kleinberg, and Éva Tardos. Maximizing the spread of influence through a social network. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 137–146. ACM, 2003.

[5] Xiaowen Dong, Dorina Thanou, Pascal Frossard, and Pierre Vandergheynst. Laplacian matrix learning for smooth graph signal representation. 2015.

[6] Aude Costard. *Estimation de la structure dindpendance conditionnelle dun rseau de capteurs. Application limagerie mdicale*. Phdthesis, University of Grenoble, Saint Martin d'Hres, France, August 2006.

[7] Jerome Friedman, Trevor Hastie, and Robert Tibshirani. Sparse inverse covariance estimation with the graphical lasso. *Biostatistics*, 9(3):432–441, 2008.

[8] MatlabToolDevelopers. Matlab-based modeling system for convex optimization. http://cvxr.com/cvx/.

[9] Aude Costard. Toolbox pour l'estimation de la structure d'indpendance conditionnelle d'un processus multivari gaussien. http://www.gipsa-lab.grenoble-inp.fr/~aude.costard/toolbox.html.

[10] Stephen Boyd, Borja Peleato Neal Parikh, Eric Chu, and James Eckstein. Matlab scripts for alternating direction method of multipliers. http://stanford.edu/~boyd/papers/admm/.